Project Report:
Protein Alignment

Ernald Shehaj
Matriculation Number: 50097030

Abstract

Proteins are large molecular chains that make most biological processes possible[1]. They make 42% of the dry weight of the human body, where they can also be found at the cellular level. We study the problem of comparing two or more DNA, RNA, or protein sequences based on their structure[2]. Investigation of protein structure similarities provides information on their functions and dynamics, as well as on the evolutionary relationship between sequences[2][1]. In this project, we compare protein structures based solely on Cα (alpha carbon). After we construct the feature vectors of each protein that integrate features like nearest-neighbour distances, residue type, secondary structures, and spatial fit, we build a cost matrix that finds the best residue-to-residue correspondence using the Hungarian algorithm. After we obtain optimal amino acid matching, we superpose the matched amino acids using the Kabsch algorithm and find the root-mean-squared-derivation (RMSD) as a quantitive measure of structural stability. After the compilation of the code we get a report of the final RMSD and the coverage, that are defined as the fraction of residues retained after pruning, to capture the quality and extent of the alignment. In addition, a visualization of the matched amino acids side-by-side is given, where the matched amino acids share the same colour. The proteins compared in this project are pig insulin and human insulin. The data that contains the features of one protein is given in a PDB file. A **PDB (Protein Data Bank) file** is a standardized plain text format for storing 3D structural data of biological macromolecules—chiefly proteins and nucleic acids, as well as their complexes [3].

1 Introduction

The structures that are responsible for the majority of the processes that make it possible for us to live are proteins. They are responsible for providing structure to cells and tissues, transporting molecules, enabling communication and movement, regulating cellular processes, and protecting the body through the immune system. They are chains of amino acids that have a function, which is determined by the 3D shape they have, therefore, shape matching is essential for understanding their function. Shape matching is the process of comparing two protein chains to identify similarities between residues (amino acids). Amino acids are molecules that serve as the building block for proteins. Figure 1 shows how an amino acid looks like.

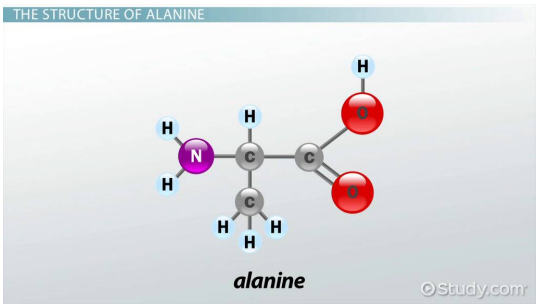


Figure 1: Chemical structure of alanine, showing the amino group (purple), carboxyl group (red), and side chain. Adapted from [4].

The similarities we may recognise while comparing two proteins give hints into: 1) the functional regions of a protein, 2) important structural characteristics of amino acids, 3) the evolutionary relationship between amino acids[2]. The approach that we do in this project is based on the Hungarian algorithm. For

every residue(amino acid) we build a feature vector, containing: 1) The Cα(alpha carbon) distance to the nearest neighbour of one residue, 2) do they contain secondary structures and if yes what do they contain, 3) what is the chemistry of the residue, 4) the distance between amino acids of different proteins using RMSD. We use the Kabsch algorithm to superpose the two structures and compute the root-mean-square deviation (RMSD) in order to have rotation invariance. However there are multiple other methods that we use in order to compare two proteins like: 1) shape retrieval methods based on histograms summarizing local or global geometrical features of the surface shape, 2) shape retrieval methods based on molecular surface maps, i.e. the projection(s) of the protein topography in the 3D space, 3) Feature-based 3D protein alignment[1]. We compare the human insuline with the insuline mollecule of the pig. The features of each protein mollecule are saved in a pdb file. The files are called respectfully 4nir.pdb and 3nir.pdb.

2 Related Works

2.1 Different categories of methods based on protein alignment

In order to grasp a better understaning at protein shape matching. We also will present how some other methods work to compare two proteins, given as PDB files.

Shape retrieval methods based on histograms summarizing local or global geometrical features of the surface shape represent protein surfaces by statistical summaries-histograms-of geometric features. These methods are responsible for the computation of descriptors such as distances, curvature, angles, or orientation distributions either globally(enitire surface) or locally(over patches), which result in a histogram that captures the proteins strctural essence. In the proces of comparing two proteins with each other the two histograms are taken and the distance between their descriptors is computed to find how similar they are. This method is computionally efficient and rotation-invariant. In Figure 2 there is an example of a histogram.[5]

Shape retrieval methods based on molecular surface maps, i.e., the projection(s) of the protein topography in the 2D space tries to represent the molecular surface of a protein by projecting its 3D topography

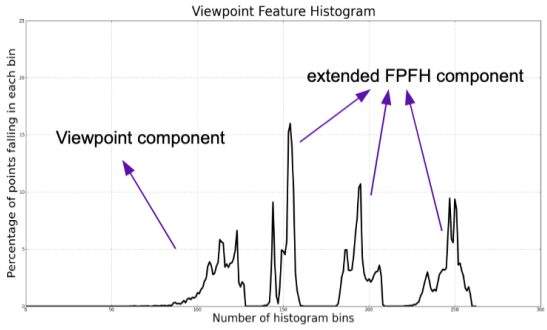


Figure 2: An example of the resultant Viewpoint Feature Histogram for a given object.[5]

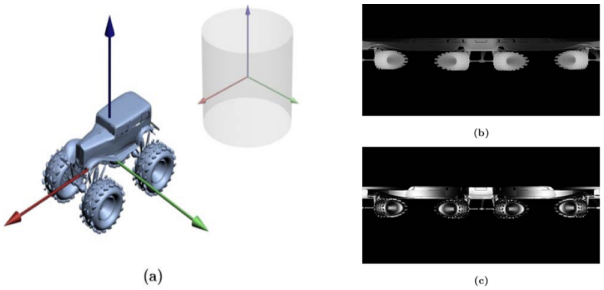


Figure 3: (a)Pose normalized 3D model, (b)the unfolded cylindrical projection of the 3D model capturing the position of the surface(the unfolded cylindrical projection of the 3D model capturing the orientation of the surface.)[6]

into a 2D panoramic map. The surface geometry of one object, like a protein, gets "unwrapped", to create a panoramic photo so to say, that captures features like curvature, depth, or surface features. The process works like this that firstly the object gets super-posed, meaning that orientation is adjusted so that the object is aligned in a consistent coordinate system. The 3D model in this case the protein gets wrapped by a virtual cylinder. After the wrapping, the surface points are projected onto the cylinder's interior. Then, after the cylinder gets "unfolded" into a 2D surface, the result is a panoramic view of the object's shape. The panoramic view that it produced can be converted into a histogram or even be encoded with descriptors. The comparison than is done by image matching-methods. In Figure 3 there is an example of the projection from 3D into a 2D surface of an object.[6]

Feature-based 3D protein alignment is the method that we implement also in our approach. The method focuses on building a feature vector for each residue of each protein and that comparing the residues to one another, with methods like in our case the hungarian algorithm.

2.2 Protein structure comparison methods

We take a deep dive in some frequently used methods such as 3D-Surfer, PANORAMA, DeepAlign, TM-Align. We take a look at some benchmarks presented by Figure 6 and understand which method would be the best to compare two proteins. The explanation of each method will be provided below.[1]

The graph shows which method is better using benchmark levels such as: protein, species, and domain.

Protein measures how well a shape matching method can compare the whole protein structure[1].

Species estimates whether proteins are correctly grouped according to the organism they come from, capturing evolutionary relationships within the same species[1].

Domain benchmark test tests the performance of these methods across forms of life with very distant evolutionary relationships, making it the most challenging[1].

In order to understand the graph, an explanation of the methods is required.

3D-Surfer is a method, where the global surface information of a protein is represented using 3D Zernike Descriptors(3DZD). These descriptors are moment-based invariants. The surface of a protein is triangulated and mapped into a 3D grid. From the mapping the 3DZD descriptors are calculated for the respective proteins. The Euclidean distance between two descriptors is used in order to assert the similarity between proteins.[1]

PANORAMA is a shape retrieval method based on molecular surface maps. The explanation is provided above.

DeepAlign is a method that uses both sequence and structural alignment to find similarities between proteins, while also using deep learning. What it uses in order to find similarities are evolutionary relationships and hydrogen-bonding similarity.[1]

TM-Align generates residue-to-residue alignment

```
HETATM  924  O   HOH D  54      -0.728   2.757  -2.902  1.00 52.70           O
HETATM  925  O   HOH D  55       7.436  20.464  15.487  1.00 64.27           O
HETATM  926  O   HOH D  56       4.515   4.955  -4.053  1.00 48.23           O
HETATM  927  O   HOH D  57      16.146   7.236 -14.395  1.00 66.22           O
HETATM  928  O   HOH D  58       0.004  -0.004 -11.812  0.33 82.86           O
HETATM  929  O   HOH D  59       2.215   3.633  -2.334  1.00 42.98           O
HETATM  930  O   HOH D  60      -5.844  14.916 -13.781  1.00 59.32           O
HETATM  931  O   HOH D  61      14.641   4.689 -10.069  1.00 40.77           O
HETATM  932  O   HOH D  62      11.550  15.703   7.081  1.00 53.44           O
HETATM  933  O   HOH D  63     -10.965  12.514 -12.677  1.00 59.24           O
CONECT   43   76
CONECT   49  223
CONECT   76   43
CONECT  154  318
CONECT  223   49
CONECT  247  826
```

Figure 4: Example snippet from a PDB file.

based on structural similarity between protein pairs independently of their sequences using heuristic dynamic programming. The scoring function is called TM-score, and the method tries to maximise it.[1]

All in all the graph compares the performance of different structural alignment and shape retrieval methods (3D-Surfer, PANORAMA, DeepAlign, TM-Align) across three benchmark levels: protein, species, and domain. Explanation of each is provided above. Moreover, it shows how well each method captures similarities at the fine-grained protein level, the evolutionary grouping at the species level, and the broad classification at the domain level. Therefore this makes it possible for the most accurate method overall to be distinguished.

3 Methods

3.1 PDB Files

Figure 4 shows how a snippet from a PDB file would look like in VS Code.

As stated above, every protein is given as PDB file, which captures every feature of the protein. The key elements of a PDB file would be:

1.HEADER provides the name of the protein, classification, deposition date, and PDB ID[3].

2.TITLE contains the protein's function, relevant experimental conditions, and provides a general description of what the PDB file contains[3].

3.COMPND describes the macromolecular components of the structure[3].

4.AUTHOR lists all the names of the researches that submitted the PDB file[3].

5.ATOM is the section of the PDB file that contains the coordinates of each atom in the macromollecule. It also contains the name of each atom, residue name,

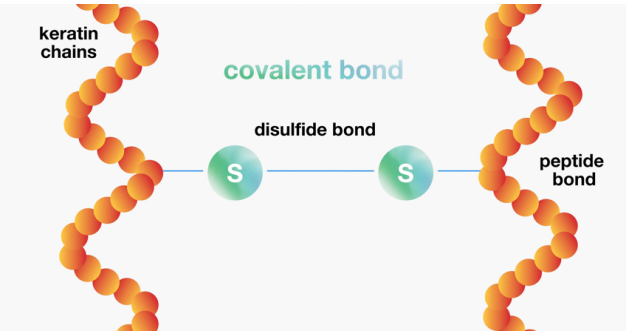


Figure 5: Disulfide bonds[9]

chain id , residue number and also occupancy.[3]

6.HETATM records in a PDB file describe non-standard atoms such as ligands, cofactors, ions, or water molecules[3]. Ligands are small molecules that bind to proteins, cofactors are non-protein compounds that assist enzyme activity, and ions are charged atoms or molecules that stabilize structures or support biological processes.

7.HELIX AND SHEET gives information on secondary structures like alpha-helixes and beta-helixes, specifying even further the residues involved[3]. Alpha-helices are spiral-shaped structures in proteins stabilized by hydrogen bonds, while beta-sheets (often called beta-helices in general talk) are sheet-like structures formed by hydrogen bonding between aligned strands[7]. Hydrogen bonding are bonds that arise due to interaction between a hydrogen atom that is bonded to two highly electronegative atoms[8].

8.CONECT lists the connectivity information in the protein structure such as covalent and non-covalent bonds, includign disulfide bridges[3]. Figure 5 shows how a dislufite brige looke like. As seen from the figure 5 each protein contains a sulphur atom that pull each other and therefore the bond is created.

3.2 PDB Files Used in This Project

Each PDB file that we work with is found on these links below:

3nir.pdb (Pig insulin): `https://www.rcsb.org/structure/1ZNI` [10]

4nir.pdb (Human insulin): `https://www.rcsb.org/structure/3I40`[11]

3.3 PDB File Parsing and Graph Construction

With the help of the article [3] we were able to produce code and therefore extract the coordinates of the individual atoms in each PDB file and render how would a protein look like. The code helps us also extract the Cα (alpha carbon) coordinates in order for us to create nodes that represent the amino acids, bonded together with peptide bonds. We use the Cα as a reference for every amino acid. Each molecule contains a central carbon (C) atom, called the α-carbon, to which both an amino and a carboxyl group are attached. Figure 1 shows each group. Each amino acid contains only one α-carbon to the centre therefore we take it a point of reference to visualize each amino acid.

We show in `3D_Shape_Matching_Ernald_Shehaj`[12] in the method called `protein_pdb_methods.py` we show how to render each atom part of the protein and also how to build a graph that represents amino acids as nodes and the peptide bonds as edges.

1.`read_pdb_to_dataframe`. The first step of our workflow involves parsing Protein Data Bank (PDB) files into a format that can be easily processed and analysed and for this purpose, we use the `read_pdb_to_dataframe` function, which leverages the BioPandas library. A Panda DataFrame is created by reading and converting the contents of a PDB file, which provides a structured representation of the atomic information. Features of the PDB file such as atom coordinates, residue names, chain identifiers, and element symbols get converted to build the DataFrame. The function also extracts header metadata from the file, which provides useful contextual information about the protein, such as methods, classification, and biological annotations.

After the function loads the PDB file into a Pandas DataFrame, the DataFrame is further processes with the help of `process_dataframe` function in our code. The function of this method nothing more than to clean up the DataFrame. Alternate atom locations(alt_loc) get resolved and the structure(DataFrame) is filtered based on the wanted level of granularity. By the selection of Cα as the level of granularity, we can serve a reduced representation of the protein backbone. Each residue (amino acid) is then assigned a unique node identifier using the `label_node_id` from the Graphein library. This ensures that the protein's structure can be

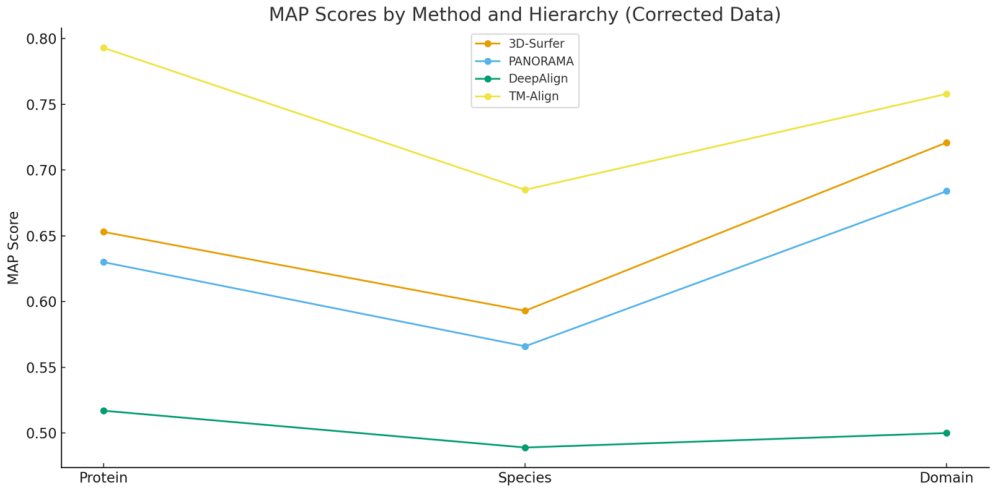


Figure 6: MAP Scores by Method and Hierarchy (corrected data). The figure compares the performance of different structural alignment and shape retrieval methods (3D-Surfer, PANORAMA, DeepAlign, TM-Align) across three levels: protein, species, and domain.

later modelled as a graph.

2.`Graph_construction`. In Figure 7 a clear visualization of the pig insulin as a graph. Each amino acid is treated as a node, and the connections between them are represented as edges. The `add_backbone_edges` function links the nodes sequentially. The function that we implemented works in the way that it sorts the residues in order of their residue number, which is contained in the PDB file, and adds edges between consecutive residues, which are called backbone bonds. In this visualization, the nodes that belong to the same chain are coloured the same. A protein is made of chains of amino acids, where in our case here we have chain A, B, C, and D of the pig insulin. Chain A for example is coloured violet. `Build_backbone_graph` is the function that creates the residue-level graph using the Graphein library. The graph is initialized with the chain identifier, then it adds nodes that represent residues from the DataFrame. The backbone edges are attached after in order to reflect the protein's structural sequence. Lastly, the `visualize_backbone_graph` function uses **Plotly** to generate a 3D interactive visualization of the graph.

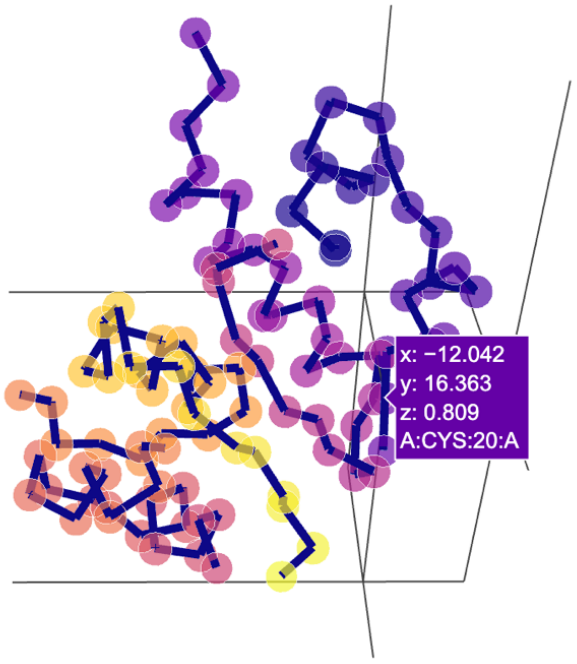


Figure 7: Backbone Protein Graph

3.`Atom_level_plotting`. In Figure 8 a clear visualization of the pig insulin as a plotting of atoms. The function `plot_atoms_with_bonds` creates an interactive 3D visualization of all atoms that are

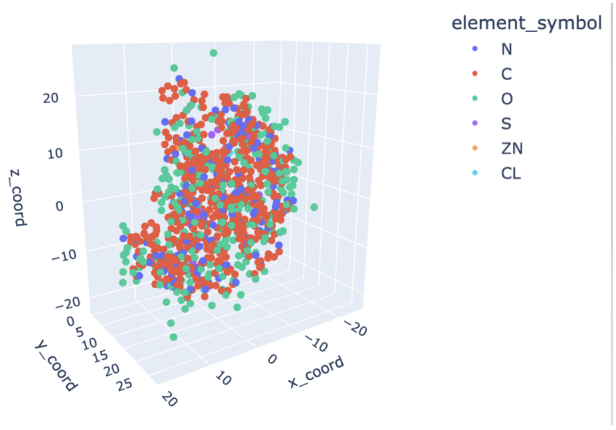


Figure 8: Plotted Atoms of the Pig Insulin

included in the PDB File. The function draws atoms as 3D markers by using their Cartesian coordinates(x_coord, y_coord, z_coord). The colouring is done by the `color_by` argument. This argument uses the argument `element_symbol` in order so that the elements part of the protein, which can be C, N, O, S appear with distinct, consistent colours. **`4.plot_two_chains_with_interface`** is the final function on our `protein_pdb_methods.py` implementation. Figure 9 provides a view of the interaction between two chains of amino acids. Chain A is shown in blue and Chain B in red. This function tries to highlight interaction between two chains by detecting and displaying in orange spatially close atoms. `plot_two_chains_with_interface` checks for atoms that are within a threshold distance of 7.0 7.0 Å[1] from one another across the two chains. These contact atoms, can indicate points such as hydrogen bonding.

3.4 Feature-based 3D protein alignment

In this section, we will talk about how we take two proteins and compare them using the Hungarian algorithm. How we build feature vectors for each residue, and how we build the matrix that makes the matching using the Hungarian method possible. This work presents nothing more than a feature based approach, which compares protein structures through Cα(alpha-carbon) atom alignment. Figure 10 shows the visualization of

[1] Å (ångström) is a unit of length commonly used in molecular biology and structural chemistry, equal to 10^{-10} meters[13].

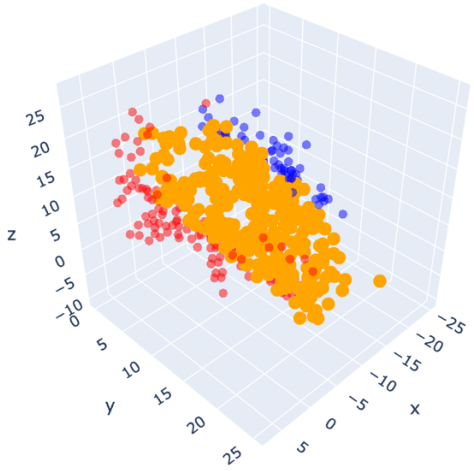


Figure 9: 1) Chain A (Blue), 2) Chain B (Red), 3) Contact Atoms (Yellow).

the comparison between the human insulin`4nir.pdb` and the pig insulin`3nir.pdb`.

3.4.1 Core Components

k-Nearest Neighbors(kNN) Fingerprinting is an algorithm that for each residue it computes the distance of each residue to its k nearest neighboring Cα atoms, where typically the k=10(10 nearest neighbours). In order to find the distance between residue i and neighbours j, we computed the Euclidean distance:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$$

The distances form a k-dimensional feature vector for each residue. When we build the cost matrix, the kNN cost, while trying to match residue i to j is calculated as the normalized Euclidean distance between their respective kNN vectors:

$$C_{knn}(i,j) = w_{knn} \times \frac{\|KNN_A(i) - KNN_B(j)\|_2}{k \times 10.0}$$

This is the first step.[14]

Secondary Structure Prediction with the usage of backbone angles tries to find the secondary structures.

$$\theta = \arccos(\mathbf{u} \cdot \mathbf{v}), \text{ where } \mathbf{u} = \frac{\mathbf{p}_1 - \mathbf{p}_2}{\|\mathbf{p}_1 - \mathbf{p}_2\|} \text{ and } \mathbf{v} = \frac{\mathbf{p}_3 - \mathbf{p}_2}{\|\mathbf{p}_3 - \mathbf{p}_2\|}$$

where $\mathbf{p}_i = [x_i, y_i, z_i]$, finds the angle between three consecutive alpha-Carbon molecules.

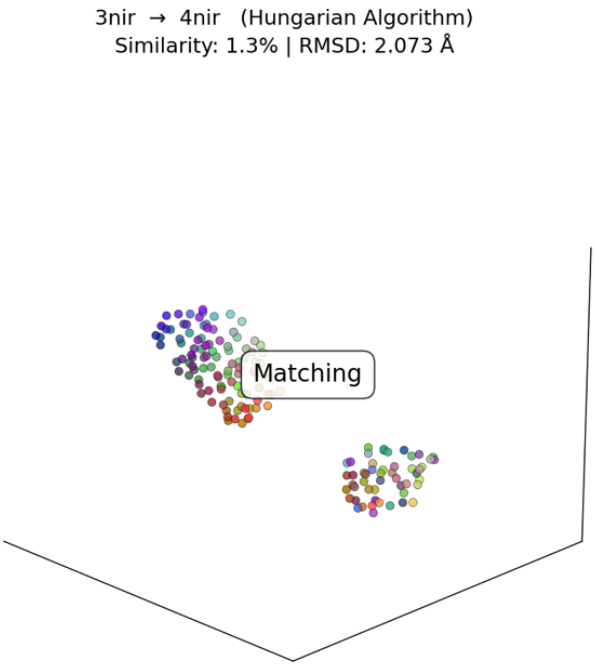


Figure 10

$\phi = \text{atan2}(y, x)$, where $x = \mathbf{v} \cdot \mathbf{w}$ and $y = (\mathbf{b}_1 \times \mathbf{v}) \cdot \mathbf{w}$

gives the dihedral angle between 4 consecutive C_α carbons with $\mathbf{b}_0 = \mathbf{p}_2 - \mathbf{p}_1$, $\mathbf{b}_1 = \mathbf{p}_3 - \mathbf{p}_2$, $\mathbf{b}_2 = \mathbf{p}_4 - \mathbf{p}_3$, and $\mathbf{v}$, $\mathbf{w}$ being projections perpendicular to $\mathbf{b}_1$. Helices are identified by $80 \leq \theta \leq 120$ and $-90 \leq \phi \leq -30$, while beta sheets show $100 \leq \theta \leq 140$ and $|\phi| \geq 160$[15, 16, 17, 18, 19, 20].

Residue Chemistry Classification Each amino acid is part of a chemical category. [12] the code of the project in the `compare_pdb_rmsd.py` it is shown that amino acids can be hydrophobic, aromatic, polar, negatively charged, and positively charged[21]. The cost matrix works in the way such as if we have identical chemistry between residues we have zero cost, same broad category receives 0.4, and different categories receive 1.0. This ensures chemical sensible correspondences.

Kabsch Algorithm is used in order to produce rotation invariance, computing the optimal matrix $\mathbf{R}$ and translation vector $\mathbf{t}$ that minimises the RMSD between two point sets P and Q in 3Dspace, following the steps[22]:

- Compute centroids of $\mathbf{P}$ and $\mathbf{Q}$
- Center both point sets by subtracting centroids
- Compute cross-covariance matrix $\mathbf{C} = \mathbf{P'}^T\mathbf{Q'}$
- Compute SVD: $\mathbf{C} = \mathbf{U\Sigma V}^T$
- Compute $d = \text{sign}(\det(\mathbf{VU}^T))$ and $\mathbf{B} = \text{diag}(1, 1, d)$
- Optimal rotation: $\mathbf{R}^* = \mathbf{VBU}^T$
- Optimal translation: $\mathbf{t}^* = \bar{\mathbf{q}} - \bar{\mathbf{p}}$

The RMSD between aligned point sets is[22]:

$$\text{RMSD} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\mathbf{q}_i - \mathbf{R}^*(\mathbf{p}_i + \mathbf{t}^*)\|^2}$$

3.4.2 Alignment Pipleline

The cost matrix is formed on this way:

$C(i,j) = w_{knn} \times C_{knn} + w_{ss} \times C_{ss} + w_{chem} \times C_{chem} + w_{rmsd} \times C_{rmsd}$

The Hungarian algorithm produces an initial matching without using Kabsch alignment and then the whole alignment is repeated when we do use Kabsch to align the residues. After alignment, poorly matched pairs with distance above 3.5 Å are removed. The similarity score combines coverage and fit quality:

$$\text{Similarity} = 100 \times \frac{\text{kept_pairs}}{\min(N_A, N_B)} \times \frac{1}{1 + \alpha \times \text{RMSD}}$$

The output includes a CSV mapping. The CSV mapping gets visualized in `visualize_match.py`.

4 Discussion

The method talked above can indeed generate a matching between proteins, but it has many flaws. Firstly, it does not check if the residues are of the same type, but it rather concentrates in their position in the 3D space. Moreover, it is not very able in error handling. If a protein has a missing residue, the algorithm here cannot skip them. Instead, it forces a one to one match. The method implemented by us is also very sensitive to noise, where a few outlier residues with bad coordinates can distort the match and affect the RMSD. Lastly, the Hungarian algorithm method is a greedy matching method that sometimes cannot implement methods that may be biologically more meaningful.

5 Conclusions

However, the code is able to perform very well on feature based comparison. It takes into consideration features that we discussed above in order to find the best matching. The usage of the Kabsch Algorithm brought rotation invariance, where it removes translation/rotation effects. In this case, we also saw that the code was able to work with different sizes of proteins, like the pig insulin and the human insulin, which have different sizes. Lastly, the code provides at the end similarity scoring and a visual output.

6 Code availability

The code developed for this report can be found in the repository provided with the report: $3D_{S}hape_{M}atching_{E}rnald_{S}hehaj$

References

[1] Mohamed Machat et al. "Comparative Evaluation of Shape Retrieval Methods on Macromolecular Surfaces: An Application of Computer Vision Methods in Structural Bioinformatics". In: *Structural Bioinformatics* (2025). Laboratoire GBCM, EA 7528, Conservatoire National des Arts et Métiers, Hesam Université, 2 rue Conté, 75003 Paris, France; Laboratoire XLIM, UMR CNRS 7252, Université de Limoges, 123 avenue Albert Thomas, 87000 Limoges, France.

[2] Geneious. *Understanding Sequence Alignment*. `https://www.geneious.com/guides/understanding-sequence-alignment`. Accessed: 2025-09-24. 2025.

[3] James Brasier. *Working with PDB files in Python*. Medium article. 2020. URL: `https://medium.com/@jgbrasier/working-with-pdb-files-in-python-7b538ee1b5e4` (visited on 09/27/2025).

[4] Study.com. *Alanine: Structure & Formula*. Figure retrieved from Study.com. n.d. URL: `https://study.com/academy/lesson/video/alanine-structure-formula-quiz.html` (visited on 09/29/2025).

[5] Radu Bogdan Rusu et al. "Fast 3D Recognition and Pose Using the Viewpoint Feature Histogram". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Taipei, Taiwan: IEEE, 2010, pp. 2155–2162. URL: `http://vigir.missouri.edu/~gdesouza/Research/Conference_CDs/IEEE_IROS_2010/data/papers/0497.pdf` (visited on 09/29/2025).

[6] Panagiotis Papadakis et al. "PANORAMA: A 3D Shape Descriptor Based on Panoramic Views for Unsupervised 3D Object Retrieval". In: *International Journal of Computer Vision* 89.2-3 (2009), pp. 177–192. DOI: `10.1007/s11263-009-0281-6`. URL: `https://dl.acm.org/doi/10.1007/s11263-009-0281-6` (visited on 09/30/2025).

[7] *Alpha-Helix and Beta-Sheet*. https://byjus.com/chemistry/alpha-helix-and-beta-sheet/. Accessed: 2025-09-28.

[8] BYJU'S. *Hydrogen Bonding*. https://byjus.com/jee/hydrogen-bonding/. Accessed: September 30, 2025.

[9] K18 Hair. *Science Class: 13 Bonds and Holistic Hair Health*. https://www.k18hair.com/blogs/consumer/science-class-13-bonds-holistic-hair-health. Accessed: September 29, 2025.

[10] Protein Data Bank. *Structure of Pig Insulin*. https://www.rcsb.org/structure/1ZNI. PDB ID: 1ZNI. Accessed: September 30, 2025.

[11] Protein Data Bank. *Structure of Human Insulin*. https://www.rcsb.org/structure/3I40. PDB ID: 3I40. Accessed: September 30, 2025.

[12] Ernald Shehaj. *3D_Shape_Matching_Ernald_Shehaj*. Version 1.0. Project and source code, developed at Rheinische Friedrich-Wilhelms-Universität Bonn. 2025.

[13] The Editors of Encyclopædia Britannica. *Angstrom (Å)*. n.d. URL: https://www.britannica.com/science/angstrom (visited on 09/30/2025).

[14] Thomas Cover and Peter Hart. "Nearest neighbor pattern classification". In: *IEEE Transactions on Information Theory* 13.1 (1967), pp. 21–27.

[15] Xavier Fernandez de la Cruz, Michael W Mahoney, and Byungkook Lee. "Discrete representations of the protein Cα chain". In: *Structure* 5.5 (1997), pp. 635–647.

[16] Maria M Flocco and Susan L Mowbray. "C alpha-based torsion angles: a simple tool to analyze protein conformational changes". In: *Proteins: Structure, Function, and Bioinformatics* 22.4 (1995), pp. 502–508.

[17] O Keskin et al. "Packing of side chains in low-resolution models for proteins". In: *Structure* 6.9 (1998), pp. 1047–1056.

[18] Pierre Grassein et al. "Curvature and torsion of protein main chain as local order parameters". In: *Journal of Biomolecular NMR* 74.4 (2020), pp. 279–296.

[19] Wikipedia contributors. *Dihedral angle*. https://en.wikipedia.org/wiki/Dihedral_angle. Accessed: 2025-09-30. 2025.

[20] Anil Korkut and Wayne A Hendrickson. "A force field for virtual-atom molecular mechanics of proteins". In: *Proceedings of the National Academy of Sciences* 106.37 (2009), pp. 15667–15672.

[21] William R Taylor. "The classification of amino acid conservation". In: *Journal of Theoretical Biology* 119.2 (1986), pp. 205–218.

[22] Hunter Heidenreich. *The Kabsch Algorithm*. Accessed: 2025-10-01. 2024. URL: https://hunterheidenreich.com/posts/kabsch-algorithm/.